

StudyOS — Architecture Analysis (Pre-Implementation)

No code yet — this is the design phase you asked for.

1. Core Problem

You have 10 courses (6 theory, 4 lab) and your academic life is scattered across notebooks, WhatsApp, LMS, screenshots, and memory. The result: forgotten homework, missed submissions, and walking into class without remembering what was taught last time.

The core problem isn't "I need a to-do list." It's: **"I need an external memory for my academic life that tells me, with almost no effort, what I need to know and what I need to do."**

Three sub-problems, specifically:

Memory loss — lecture content and teacher emphasis evaporate between classes.

Deadline blindness — many small deadlines across many courses, easy to lose track of urgency vs. workload.

Cognitive overload — even if all the data exists, if it's not prioritized, seeing "everything" is as bad as seeing nothing.

Everything else in your spec (labs, timetable, inbox, AI, etc.) is in service of these three problems.

2. Most Important MVP Features

Ranked by how directly they solve the core problem above:

Courses + Timetable — the skeleton everything else hangs off.

Today Dashboard — the single screen that answers "what do I have today."

Tasks with due dates + urgency levels — solves deadline blindness.

Lecture Capture (2-minute form) + Quick Review — solves memory loss; this is your most novel/valuable feature.

Course Pages — a home for each course's lectures/tasks/labs.

Lab Pipeline Tracking — labs have a different lifecycle than homework; needs its own simple state machine.

Quick Add (manual form first, not AI) — low-friction capture.

Search — "I remember the content, not where I put it."

Local persistence — the app is useless if closing the tab loses data.

Everything else (calendar, study sessions, inbox, exam prep, AI, notifications) is real, but not MVP. See §12.

3. Recommended Technology Stack

Layer	Choice
UI	React + TypeScript + Vite
Styling	Tailwind CSS
State	React hooks + Context (no Redux)
Local database	IndexedDB via Dexie.js
Search	Dexie queries + Fuse.js for fuzzy text search
Dates	date-fns
Backup	JSON export/import (built on top of Dexie)
Desktop packaging (later)	Tauri, wrapping the same web app
Backend server	None in MVP

4. Why This Stack

No backend server in the MVP. Your spec explicitly asks for local-first + desktop-later + "don't introduce unnecessary technologies." A server (Node/Express + a networked DB) adds a whole extra layer you'd have to run, understand, and keep in sync with the frontend, for zero benefit on a single-user app. Instead:

IndexedDB (via Dexie.js) is a real structured, indexed, queryable database *built into the browser*. It works fully offline, requires no setup, and Dexie gives it a friendly, promise-based API instead of raw IndexedDB (which is notoriously painful). This directly satisfies your "local-first" and "offline" requirements without extra infrastructure.

One codebase, one language. Everything is TypeScript. You don't context-switch between a frontend language and a backend language, or debug a network layer between them.

React + Vite is the most-documented frontend stack in existence — huge learning value for a CS student, fast dev server, and it's exactly what Tauri expects later.

Tailwind avoids you hand-rolling CSS for 15+ screens, while keeping styling readable and inline with the markup (good for understanding *and* speed).

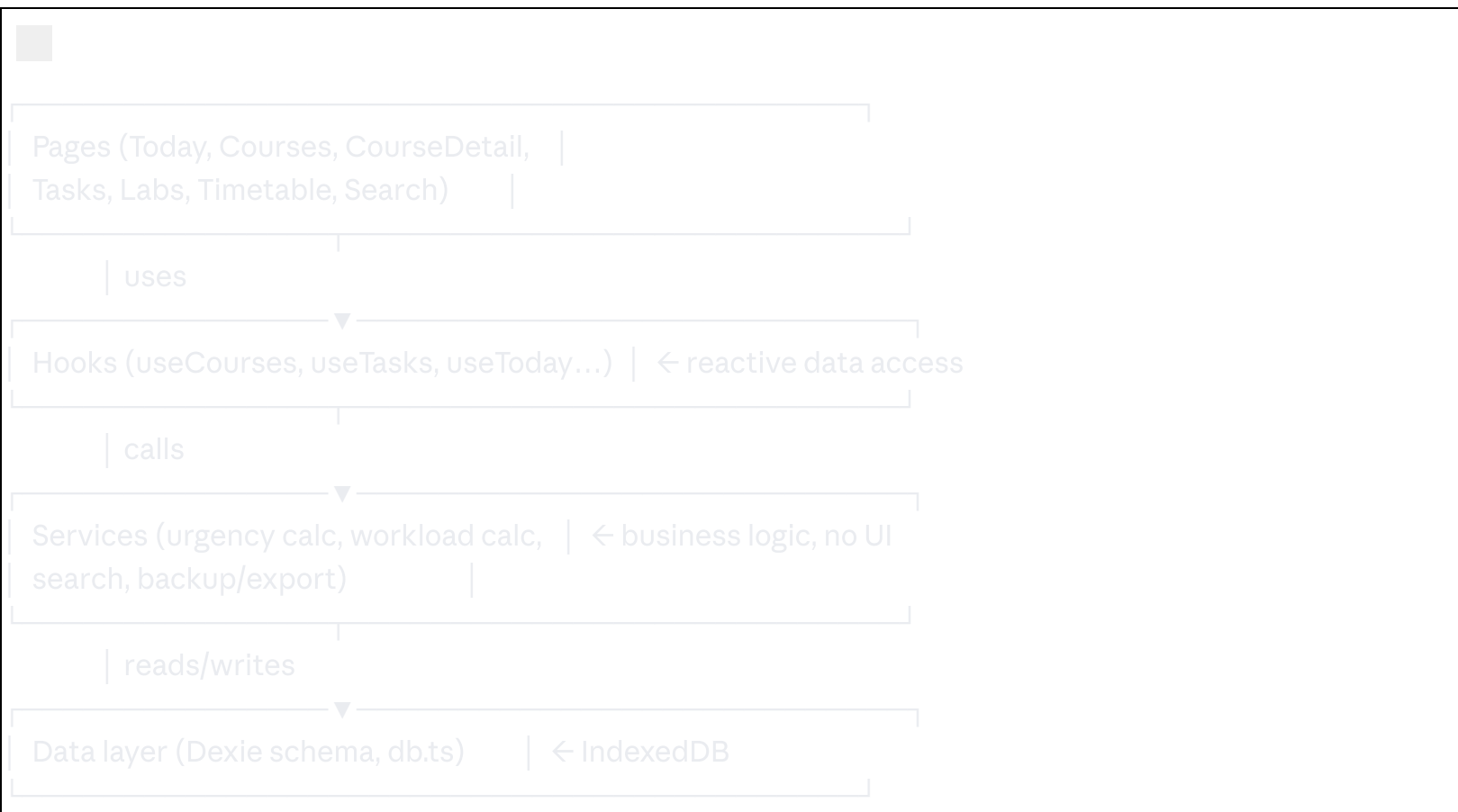
Tauri (not Electron) for desktop, later. Tauri wraps your existing web app in a native window. Its webview still runs IndexedDB, so **you don't need to rewrite your data layer to go from web to desktop** — this is the key reason your "same core app, packaged as desktop later" requirement is satisfied cleanly. Tauri also

produces much smaller, faster apps than Electron. This step is deliberately deferred — you build and use the web app first.

What I'm deliberately avoiding: a backend framework, an ORM, authentication, Redux, a UI component library, and AI dependencies. All of these are real tools, but none of them are necessary to solve your actual problem, and each one is something you'd have to learn and maintain for no return in the MVP.

5. High-Level Architecture

Single-page app, layered so each piece has one job:



Why this shape matters for you specifically: the "urgency" and "what am I forgetting" logic (§8, §18, §29 of your spec) is genuinely non-trivial. Keeping it in a `services/` folder, separate from UI components, means you can test and reason about "is this task urgent?" without touching any React code — and if you ever swap IndexedDB for SQLite (desktop), only the data layer changes.

6. Database Design

All tables live in IndexedDB, defined via a single Dexie schema file. MVP tables (Phase 2+ tables noted separately):

MVP tables

`courses` — id, name, type (theory/lab), instructor, color, createdAt

`classSchedule` — id, courseId, dayOfWeek, startTime, endTime, room, instructor

`lectures` — id, courseId, date, topics, importantConcepts, teacherEmphasis, homeworkText, rememberForNext, confusingNotes, mentionedQuizExam, reviewed (bool)

`tasks` — id, courseId, title, type, description, dueDate, dueTime, estimatedMinutes, priority, status, progress, submissionMethod, submissionLink, notes

`subtasks` — id, taskId, title, completed

`labs` — id, courseId, labNumber, date, taskDescription, pipelineStage (given/understanding/implementation/testing/report/submission/viva), deadline, submissionStatus

Phase 2 tables (schema designed now, built later): `exams`, `studySessions`, `notes`, `resources`, `reminders`, `questions`, `submissions`, `inboxItems`.

Designing all tables' shapes now (even unused ones) avoids painful schema migrations later — but only MVP tables get UI and logic in Phase 1.

7. Main Entities and Relationships



Everything hangs off `Course`. There is intentionally **no `User` table** in the MVP — this is a single-user personal app; adding auth/multi-user is explicitly out of scope (§12).

8. Application Pages / Screens (MVP)

Today — dashboard: today's classes, next class + last-lecture card, urgent tasks, top 3–5 actions.

Courses — list of all 10 courses with quick health indicators.

Course Detail — tabs: Overview / Lectures / Tasks / Labs.

Tasks — full task list, filterable by course/status/priority.

Labs — lab list across all courses, grouped by pipeline stage.

Timetable — editable weekly schedule.

Search — global search results.

Lecture Capture (modal, triggered from "mark class complete").

Quick Review (modal, triggered from "upcoming class").

Quick Add (global modal, accessible from anywhere).

9. Main User Flows

Morning flow: Open Today → see today's classes + urgent tasks + top actions (matches your §36 mockup exactly).

Post-class flow: Mark class complete → Lecture Capture form (all fields skippable) → saved → optionally spins off a Task if homework was mentioned.

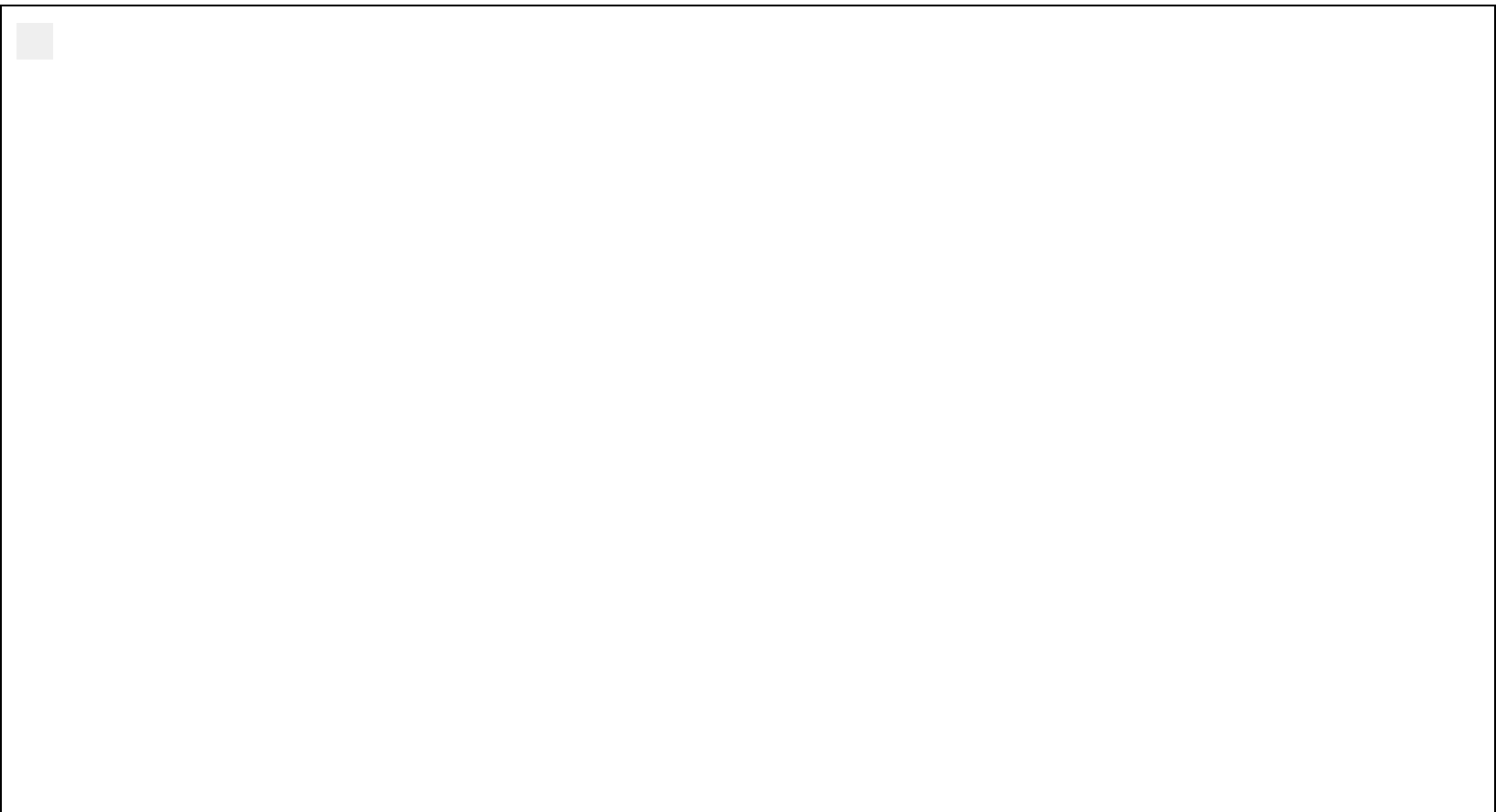
Pre-class flow: Today dashboard detects the next class → shows Quick Review card pulling the last lecture for that course → "Mark Reviewed."

Task capture flow: Quick Add → pick "Task" → fill minimal form (course, title, due date) → save. (Natural-language parsing is Phase 3, not MVP — see §12.)

Lab flow: Open a lab → move it through pipeline stages (Given → Understanding → Implementation → Testing → Report → Submission → Viva) via a simple stepper.

Search flow: Type a query → results grouped by type (lectures, tasks, labs, notes).

10. Folder / Project Structure



```
studynos/
├── src/
│   ├── main.tsx
│   ├── App.tsx
│   ├── db/
│   │   ├── schema.ts    (Dexie table definitions)
│   │   └── db.ts        (Dexie instance)
│   ├── types/
│   │   └── index.ts      (TypeScript interfaces for each entity)
│   ├── services/
│   │   ├── urgency.ts    (due-date + workload → urgency level)
│   │   ├── workload.ts   (weekly workload estimation)
│   │   ├── search.ts
│   │   └── backup.ts     (JSON export/import)
│   ├── hooks/
│   │   ├── useCourses.ts
│   │   ├── useTasks.ts
│   │   ├── useLectures.ts
│   │   ├── useLabs.ts
│   │   └── useToday.ts
│   ├── components/      (Button, Card, Modal, StatusBadge, etc.)
│   ├── pages/
│   │   ├── Today/
│   │   ├── Courses/
│   │   ├── CourseDetail/
│   │   ├── Tasks/
│   │   ├── Labs/
│   │   ├── Timetable/
│   │   └── Search/
│   └── utils/
│       ├── dateUtils.ts
│       └── formatters.ts
├── index.html
├── package.json
└── vite.config.ts
```

No `server/`, `api/`, or `backend/` folder in the MVP — deliberately.

11. Development Phases

Phase 0 — Foundation: project scaffold, Dexie schema, Course + Timetable CRUD (no dashboard logic yet).

Phase 1 — Core loop: Today dashboard (classes only), Tasks CRUD, urgency levels (§8 of your spec).

Phase 2 — Memory system: Lecture Capture, Quick Review, Course Detail pages.

Phase 3 — Labs: Lab pipeline tracking + lab list view.

Phase 4 — Capture & find: Quick Add (manual form), global Search.

Phase 5 — Awareness: Weekly overview, course health indicators, "What am I forgetting?" scan.

Phase 6 (your spec's Phase 2): Calendar, study sessions, inbox, questions, exam prep, reminders/notifications, backup/export UI.

Phase 7 (your spec's Phase 3): Tauri desktop packaging, then optionally AI features (NLP quick add, lecture summarization).

Each phase ends with something you can actually use daily — that's intentional, matching your own "acceptance test" scenarios.

12. Features NOT Built Initially

Natural-language / AI-parsed Quick Add (start with a plain form)

Push notifications / reminders (real ones need a service worker + permissions; noisy to get right early)

Calendar month/week visual view

Study session (Pomodoro-style) tracking

Academic Inbox

Questions/doubts tracker

Exam preparation module

Course health visual bars (needs data history to be meaningful anyway)

File attachments with real storage (start with links only; IndexedDB isn't great for large binary files)

Multi-device sync, authentication, multi-user support

Desktop packaging (build and validate the web app first)

None of these are "wrong" — they're just not what makes-or-breaks the core problem, and building them early risks the classic trap of a huge spec never shipping anything usable.

13. Potential Technical Problems & Mitigations

Problem	Mitigation
IndexedDB's raw API is awkward/async-heavy	Dexie.js abstracts this into a clean, promise-based API; use <code>liveQuery</code> for auto-updating UI

Problem	Mitigation
Browser storage can be cleared by the user/OS	Ship JSON export/import from day one (§40 of your spec); nudge periodic backups; desktop version (Tauri) later gets real file-based persistence
Date/timezone bugs in due dates & urgency calc	Store dates as ISO strings, use <code>date-fns</code> consistently, write urgency logic as pure functions you can unit test
Search gets slow as data grows	Use indexed Dexie fields for filtering first, then run Fuse.js only on the already-narrowed subset
Urgency/workload algorithm becomes over-engineered	Start with a simple rule table (days-until-due × estimated-hours → level), iterate only if it's actually wrong in practice
Scope creep (this spec is huge)	Treat §11 phases as hard gates — don't start Phase N+1 until Phase N is something you use for a real week
Moving from IndexedDB (web) to SQLite (desktop) later, if ever needed	The <code>services/</code> and <code>hooks/</code> layers never touch IndexedDB directly — only <code>db/</code> does, so a storage swap is isolated to one folder

Next Step

This is the design. **I'm waiting for your approval** (or requested changes) before writing any implementation code, per your process. Once approved, I'd start at Phase 0: project scaffold + Dexie schema + Course/Timetable CRUD.